

The background is a stylized illustration of a research laboratory. Several diverse individuals are shown working at computer workstations. In the background, a complex network diagram is overlaid, featuring various nodes and connecting lines. Some nodes are labeled with text in different languages, including '1,000', 'VERSION', and '40009-35 + 6D'.

Version Control for Researchers

Richard Polzin (19.08.2026)



Table of Contents

Today we'll cover the fundamentals of version control and Git — from your first commit to collaborating with others.

1. Version Control for Researchers
2. Table of Contents
3. ✨ Version Control ✨
4. But I Work Alone...
5. Git Workshop
6. Initial Setup
7. The Basic Workflow
8. Example
9. Recap
10. Branching and Merging: The Idea
11. Branching and Merging: In Practice
12. Quick Context Switch: git stash
13. Merge Conflicts
14. Best Practices (for Researchers)
15. From Tag to Citation: Zenodo
16. Common Pitfalls

✨ Version Control ✨

What?

- A system for managing changes to files over time
- Allows simultaneous work on the same project
- A history of changes and the ability to revert
- Logically separate features 🍷

Why?

- Makes collaboration easier 🏆
- Who deleted my files? Where is my main.py??
- Tracking changes and ensuring reproducibility
- Avoiding "final_version_v3_revised_FINAL.py" 🙄
- Backup and restoring previous versions

How?

Git of course!

- A distributed version control system (VCS)
- Tracks changes in code and text files
- Enables collaboration across different versions
- Supports parallel development
- Provides a detailed history of changes for accountability

But I Work Alone...

- 🕒 **Time travel for your own work** — ever broken something and not known what changed? `git diff` and `git log` answer that instantly
- ✏️ **Safe experimentation** — try a completely different approach on a branch; if it fails, delete it and you're back to where you started
- 🏷️ **Reproducibility** — tag the exact code state used for a paper submission; re-run it a year later and get the same results
- 💾 **Free offsite backup** — push to GitHub/GitLab and your work survives a stolen or dead laptop
- 🤝 **Future-you is a collaborator too** — good commit messages are messages to yourself six months from now

Git Workshop

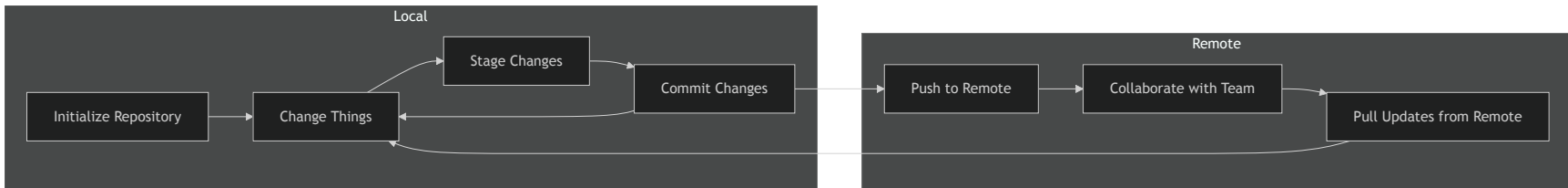
Key Concepts

- **Repository (Repo):** A directory containing all project files and history
- **Commit:** A snapshot of changes
- **Branch:** Parallel versions of the repository
- **Merge:** Combining different branches
- **Remote:** A repository hosted elsewhere (e.g., GitHub, GitLab)

Initial Setup

```
# Install Git
sudo apt install git # Linux
brew install git # macOS
choco install git.install # Windows

# Configure Git
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```



Not a CLI person? GitHub Desktop or your editor's built-in Git panel (e.g. VS Code) cover the same workflow with buttons instead of commands — this workshop uses the command line because it's the same everywhere.

The Basic Workflow

- **Initialize Repository:** Start a new repository with `git init`.
- **Make Changes:** Modify files in your working directory.
- **Stage Changes:** Select which changes to include with `git add`. Git requires an explicit staging step — only staged files go into the next commit.
- **Commit Changes:** Save a snapshot of staged changes with `git commit`.
- **Push to Remote:** Upload your commits to a remote repository with `git push`.
- **Pull Updates:** Fetch and integrate changes from the remote repository with `git pull`.

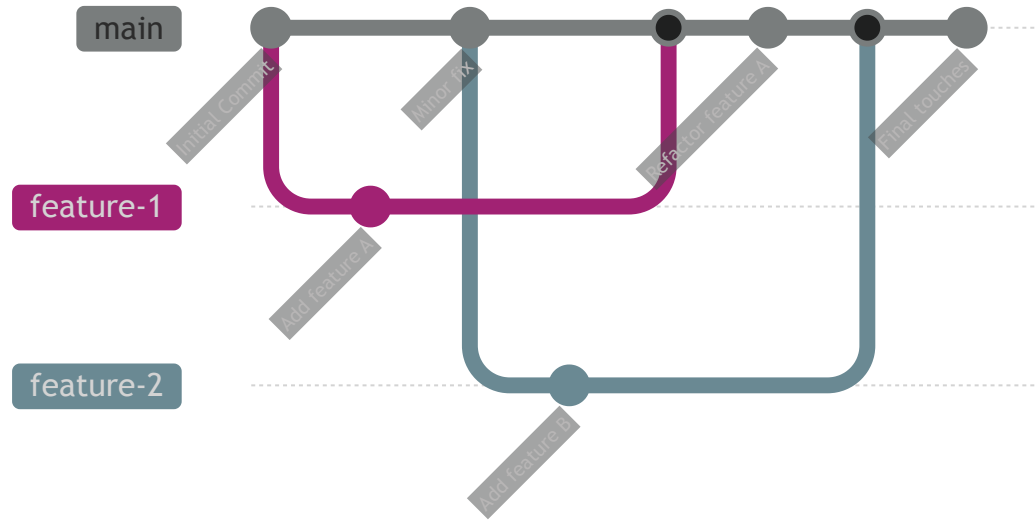
Example

```
27  $ git diff
28  > diff --git a/README.md b/README.md
29  > index 22c86a3..0628ec3 100644
30  > --- a/README.md
31  > +++ b/README.md
32  > @@ -1 +1,3 @@
33  >  # My Research Project
34  > +
35  > +this is my cool description.
36  $ # Check the commit history
37  $ git log
38  > commit 0127a4e6b03cec81c38391dc643f50fdfee75f4b (HEAD -> main)
39  > Author: Your Name <you@example.com>
40  > Date:   Mon Aug 17 13:37:57 2026 +0100
41  >
42  >   Initial commit
```

Recap

- **Version Control:** Manage changes to files over time, enable collaboration and track history.
- **Git:** The (coolest 😊) software to do version control with.
- **Key Concepts:** Repository, Commit, Branch, Merge, Remote.
- **Basic Commands:** `git init`, `git add` (stage), `git commit` (snapshot), `git status`, `git log`, `git diff`.

Branching and Merging: The Idea

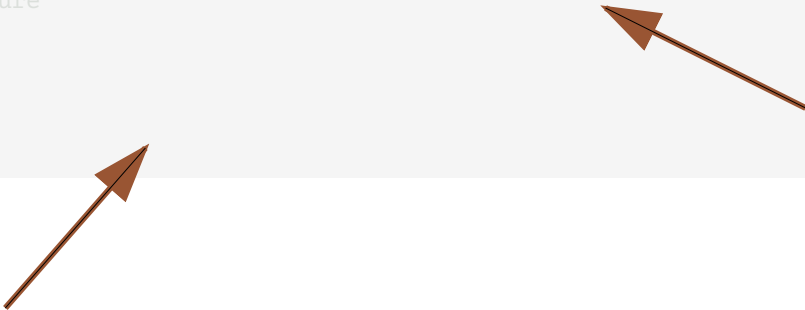


Branching and Merging: In Practice

- **Isolation:** Work on different features or fixes separately from the main codebase.
- **Parallel Development:** Enable team members to work on separate features simultaneously.
- **History Tracking:** Maintain individual commit histories for easy tracking and reversion.
- **Merging:** Combine changes from different branches back into the main codebase.
- **Experimentation:** Safely test new ideas without affecting the stable codebase.

```
# Create and switch to a new branch (modern syntax)
git switch -c new_feature # older: git checkout -b new_feature

# Merge changes back to main branch
git switch main
git merge new_feature
```



Quick Context Switch: git stash

Need to switch branches but you're mid-change and not ready to commit? Don't panic-commit — stash it.

- `git stash` — shelves your uncommitted changes, gives you a clean working directory
- `git switch other-branch` — do whatever you needed to do
- `git switch -` back, then `git stash pop` — restores your changes and removes them from the stash

```
$ git stash
> Saved working directory and index state WIP on main: a1b2c3d Add feature
$ git switch main
$ git switch -
$ git stash pop
> On branch new_feature
> Changes not staged for commit: ...
```

`git stash list` shows all shelved stashes — you can stash more than once.

Merge Conflicts

When two branches change the same lines, Git can't merge automatically.

```
$ git merge new_feature  
> CONFLICT (content): Merge conflict in analysis.py  
> Automatic merge failed; fix conflicts and then commit the result.
```

Git marks the conflicting section — edit the file to resolve it:

```
- threshold = 0.5    # your version (main)  
+ threshold = 0.8    # incoming version (new_feature)
```

- Edit the file to keep what you want and remove the markers
- `git add analysis.py` — mark it resolved
- `git commit` — complete the merge
- `git status` shows which files still have conflicts

Collaborating with Remotes

- **Remote Repository:** A version of your project hosted on the internet.
- **Push/Pull:** Upload/Download changes to and from remote.
- **Origin:** The name of the remote repository.
- **Clone:** Create a local copy of a remote repository.

GitHub, GitLab, and Alternatives



- **GitHub:** Most popular for open-source projects; large community and integrations
- **GitLab (RWTH):** Available for RWTH members at git.rwth-aachen.de — use your TIM credentials
- **GitLab-CE:** Self-hosted open-source version; fewer features but no vendor lock-in
- **Codeberg / Forgejo:** Community-run, fully open-source alternatives

Best Practices (for Researchers)

-  **Meaningful commit messages:** "Fix threshold in model A" beats "fixed stuff"
-  **Branch per experiment:** keep `main` clean; use branches for new ideas so you can always go back
-  **Tag paper versions:** `git tag v1.0-submission` so you can reproduce results from a specific submission
-  **Push regularly:** remote = free backup; don't lose a week of work to a dead laptop
-  **Version configs alongside code:** commit your hyperparameter files and experiment configs, not just the scripts
-  **Use `.gitignore`:** keep large data and temp files out of the repo

```
# .gitignore for a typical Python research project
data/                # large datasets - use DVC or similar instead
*.h5                 # model checkpoints
*.pkl
__pycache__/_
.ipynb_checkpoints/
.env                 # secrets / API keys - never commit these
```

From Tag to Citation: Zenodo

A `git tag` makes a submission reproducible on your machine — Zenodo makes it citable by anyone.

- Connect your GitHub repo to zenodo.org (free, one-time setup)
- Every new GitHub **release** is automatically archived and gets a permanent **DOI**
- Cite the exact code version behind your results — in the paper, not just "code available on request"
- GitLab repos: archive manually via Zenodo's upload form

Common Pitfalls

- 🐘 **Committing large files by accident** — a 2 GB dataset in git history is permanent and painful to remove; set up `.gitignore` before the first commit
- 🤖 **Detached HEAD** — happens when you `git checkout` a commit hash directly; you're not on any branch, so commits get lost. Fix: `git switch main`
- 🔑 **Committing secrets** — API keys, passwords, `.env` files pushed to a public repo are compromised immediately; always `.gitignore` them
- 💣 **Force-pushing shared branches** — `git push --force` rewrites history for everyone on the team; avoid on `main`, use `--force-with-lease` if you must
- 📝 **Vague commit messages** — "fixed it" or "update" tells future-you nothing; a message like "Fix off-by-one in sliding window" is searchable and self-documenting
- 🛠️ **Working directly on main** — one bad commit blocks everyone; use branches even when working alone
- ↻ **Line-ending chaos (CRLF vs LF)** — Windows and Mac/Linux collaborators can turn every line of a file into a "change"; add a `.gitattributes` with `* text=auto` to normalize it repo-wide

Advanced Topics

- **Git LFS** — store large files (datasets, model weights) without bloating the repo: `git lfs track "*.h5"`
- **nbstripout** / **nbdime** — strip notebook outputs before committing so diffs stay readable
- **CI/CD** — automatically run tests or re-run analysis on every push (GitHub Actions, GitLab CI)
- **Submodules** — reference another repo inside yours; useful for shared libraries across projects
- `git bisect` — binary-search your commit history to find exactly which commit broke something
- **SSH keys** — stop typing your password on every push: `ssh-keygen` + add public key to GitHub/GitLab

Try It Yourself

```
# Create a repo and make your first commit
git init my_research && cd my_research
echo "# My Project" > README.md
git add README.md && git commit -m "Initial commit"

# Branch, change, merge
git switch -c add-description
echo "This project does X." >> README.md
git add README.md && git commit -m "Add project description"
git switch main && git merge add-description

# Explore the history
git log --oneline
```

Push it to git.rwth-aachen.de or GitHub to get a free remote backup!

Summary

- Version Control helps manage research projects efficiently
- Enables collaboration and reproducibility
- Learning basic Git commands is a valuable skill
- Use hosted platforms for better sharing and tracking
- Start early, commit often, write good commit messages



Questions?

Richard Polzin

richardpolzin.com · rpolz@ukaachen.de · git.rwth-aachen.de